OPERATING SYSTEM (CT 656)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 7: Deadlock

Syllabus Hours: 5 hrs | Marks Weight: 10 marks

Complete Exam-Ready Answers | All Past Questions Covered (2070–2082)

CHAPTER 7 SYLLABUS TOPICS

- 7.1 Principles of Deadlock (Necessary conditions, Resource Allocation Graph)
- 7.2 Deadlock Prevention (Attack each necessary condition)
- 7.3 Deadlock Avoidance (Banker's Algorithm, Safe State)
- 7.4 Deadlock Detection (Detection Algorithm, Wait-for Graph)
- 7.5 Recovery from Deadlock (Process termination, Resource preemption)
- 7.6 An Integrated Deadlock Strategy
- 7.7 Other Issues: Two-phase locking, Communication Deadlock, Livelock, Starvation

Prepared by: Santosh Kushwaha (ACE079BCT)

SECTION 1: Deadlock — Definition, Conditions & Resource Allocation Graph

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2080 Chaitra, 2079 Chaitra, 2079 Ashwin, 2078 Poush, 2078 Chaitra, 2079 Jestha

Q1. Define Deadlock. What are the necessary conditions for deadlock? How can deadlock be handled? Explain. [3+8]

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra

Answer:

Definition of Deadlock

A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting to acquire a resource held by another process in the same set — creating a circular wait from which no process can proceed.

Formal Definition: A set of processes P = {P1, P2, ..., Pn} is in deadlock if and only if every process Pi in P is waiting for a resource that can only be released by some other process Pj in P.

Four Necessary Conditions for Deadlock (Coffman Conditions, 1971)

ALL four conditions must hold simultaneously for deadlock to occur:

Condition	Description	Example
1. Mutual Exclusion	At least one resource must be held in a non-sharable mode — only one process can use it at a time	A printer cannot be used by two processes simultaneously
2. Hold and Wait	A process holds at least one resource while waiting to acquire additional resources held by other processes	Process P1 holds R1 and waits for R2 which P2 holds
3. No Preemption	Resources cannot be forcibly taken from a process; a process must voluntarily release resources it holds	OS cannot forcibly take a mutex from a process
4. Circular Wait	A circular chain of processes exists: P1 waits for P2, P2 waits for P3, ..., Pn waits for P1	P1→R2→P2→R3→P3→R1→P1 forms a cycle

Deadlock Handling Strategies

Strategy	Approach	Characteristic
1. Prevention	Eliminate one of the 4 necessary conditions proactively	Never allows deadlock — restrictive, wastes resources
2. Avoidance	Dynamically check if granting a resource leads to unsafe state; use Banker's Algorithm	Requires advance knowledge of max resource needs
3. Detection & Recovery	Allow deadlock to occur; periodically detect it; then recover (kill process or preempt resources)	Less restrictive but costly recovery
4. Ignorance (Ostrich Algorithm)	Ignore the problem — assume deadlock is rare enough not to matter	Used in most general-purpose OS (Linux, Windows)

Resource Allocation Graph (RAG)

A directed graph used to describe the state of resources and processes:

- Process node: Circle (P1, P2, ...)
- Resource node: Rectangle with dots — each dot is one instance (R1●, R2●●)
- Request edge: Pi → Rj (process Pi requests resource Rj)
- Assignment edge: Rj → Pi (resource Rj is assigned to process Pi)

Deadlock Detection using RAG:

- If the RAG has NO CYCLE → No deadlock
- If the RAG has a CYCLE AND each resource has only one instance → Deadlock GUARANTEED
- If the RAG has a CYCLE but some resources have multiple instances → Deadlock POSSIBLE (need further analysis)

Q2. What is deadlock and livelock? Explain the necessary conditions for deadlock. A system has 2 processes and 3 resource types. Is deadlock possible? Explain. [2+3+4]

Exam Ref: 2081 Ashwin, 2079 Chaitra

Answer:

Deadlock vs Livelock vs Starvation

Feature	Deadlock	Livelock	Starvation
Process State	Blocked — cannot proceed	Active but making no progress	Waiting indefinitely
Resource State	Resources held and requested in cycle	Resources released and re-requested	Resources given to others repeatedly
Resolution	External intervention required	Change in process behavior	Aging / priority adjustment
Example	Two trains on same track, neither can move	Two people both step aside simultaneously to let each other pass — forever	Low-priority process never gets CPU
Detection	Cycle in wait-for graph	CPU usage 100% but no progress	Process wait time grows indefinitely

Can Deadlock Occur with 2 Processes and 3 Resource Types?

YES, deadlock is possible. Example:

- P1 holds R1, requests R2
- P2 holds R2, requests R1
- P1 and P2 are now in a circular wait — deadlock!

The minimum configuration for deadlock: 2 processes, 2 resource types (1 instance each). Having 3 resource types provides more ways for deadlock to form but does not prevent it.

However: If the TOTAL number of resources ≥ (number of processes + 1), deadlock can be prevented. With 2 processes, if total R ≥ 3 instances, deadlock is impossible for single-resource-type systems.

SECTION 2: Deadlock Prevention & Avoidance

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2080 Chaitra, 2079 Jestha, 2079 Chaitra, 2078 Poush, 2079 Ashwin

Q3. Explain Deadlock Prevention techniques. How can each of the four necessary conditions be attacked? [5]

Exam Ref: 2082 Kartik, 2080 Ashwin, 2079 Chaitra, 2079 Jestha

Answer:

Deadlock Prevention — Attack Each Condition

Deadlock prevention ensures the system NEVER enters a deadlock by ensuring at least one of the four necessary conditions can never hold:

Condition to Attack	Prevention Method	Drawback
1. Attack Mutual Exclusion	Make resources sharable — multiple processes can use simultaneously. Use read/write locks for read-sharing.	Not possible for all resources (e.g., printers MUST be mutually exclusive)
2. Attack Hold & Wait	Method A: Process requests ALL resources before starting execution. Method B: Release all held resources before requesting new ones.	A: Low resource utilization, starvation possible. B: Data integrity issues during re-request
3. Attack No Preemption	If a process holding resources requests another that cannot be immediately allocated, release all its held resources. Add to waiting queue; resume only when ALL needed resources available.	Not suitable for non-preemptable resources (printers, files)
4. Attack Circular Wait	Assign a global ordering number to all resource types. Processes must request resources in strictly increasing order of type number.	May force suboptimal request ordering; implementation overhead

Key Insight: Prevention is simple but overly conservative — it reduces system throughput significantly. Most practical systems prefer avoidance or detection.

Q4. What is Safe State? What is Banker's Algorithm for Deadlock Avoidance? Explain with the data structures used. [2+6]

Exam Ref: 2081 Chaitra, 2080 Chaitra, 2079 Ashwin, 2078 Poush

Answer:

Safe State vs Unsafe State vs Deadlock

Safe State: A state is SAFE if there exists at least one 'safe sequence' — an ordering of all processes such that each process can complete by using the currently available resources plus the resources held by processes that have already completed.

Unsafe State: A state where no safe sequence exists — does NOT necessarily mean deadlock, but deadlock COULD occur.

Relationship: Safe State □ Unsafe State □ Deadlock (deadlock is a subset of unsafe states).

Banker's Algorithm — Data Structures

Let n = number of processes, m = number of resource types.

Data Structure	Size	Meaning
Available[m]	1D vector of size m	Available[j] = number of available instances of resource type Rj
Max[n][m]	2D matrix n×m	Max[i][j] = max instances of Rj that process Pi may need
Allocation[n][m]	2D matrix n×m	Allocation[i][j] = instances of Rj currently allocated to Pi
Need[n][m]	2D matrix n×m	Need[i][j] = Max[i][j] − Allocation[i][j] (remaining need)
Work[m] (temporary)	1D vector	Work = Available initially; updated as processes finish
Finish[n] (temporary)	Boolean array	Finish[i] = false initially; true when Pi can complete

Safety Algorithm (Finding Safe Sequence)

```
1. Work = Available;  Finish[i] = false for all i
2. Find index i such that:
      a) Finish[i] == false
      b) Need[i] <= Work  (can Pi complete with current work?)
   If no such i exists → go to step 4
3. Work = Work + Allocation[i]   (Pi finishes, releases its resources)
   Finish[i] = true
   Go to step 2
4. If Finish[i] == true for ALL i → SAFE STATE
   Else → UNSAFE STATE
```

Resource Request Algorithm (for process Pi requesting Requesti)

```
1. If Requesti <= Need[i]: proceed; else ERROR (exceeded max claim)
2. If Requesti <= Available: proceed; else Pi must WAIT
3. Pretend to allocate resources:
      Available = Available - Requesti
      Allocation[i] = Allocation[i] + Requesti
      Need[i] = Need[i] - Requesti
4. Run Safety Algorithm:
      If SAFE  → grant resources (actual allocation)
      If UNSAFE → rollback (restore old state), Pi waits
```

SECTION 3: Banker's Algorithm — Fully Solved Numerical

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2080 Chaitra, 2079 Ashwin, 2079 Jestha, 2078 Chaitra, 2078 Poush — MOST REPEATED NUMERICAL

Q5. How can deadlock be prevented? Consider a system with 5 concurrent processes (P0, P1, P2, P3, P4) and 4 resource types (R0, R1, R2, R3). The total instances of each resource type in the system are 3, 17, 16, 12 respectively. Allocation table and maximum claim are as follows: (a) Create the Need matrix. (b) Is the system in safe state? If so, show the safe execution sequence. [2+8]

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2080 Chaitra

Answer:

Given Data

Process	R0	R1	R2	R3	R0	R1	R2	R3	R0	R1	R2	R3
	Allocation (Gray)				Maximum (Blue)				Need = Max-Alloc (Green)			

Step 1: Given Allocation and Maximum Tables

Process	Alloc R0	Alloc R1	Alloc R2	Alloc R3	Max R0	Max R1	Max R2	Max R3
P0	0	1	1	0	0	2	1	0
P1	1	2	3	1	1	6	5	2
P2	1	3	6	5	2	3	6	6
P3	0	6	3	2	0	6	5	2
P4	0	0	1	4	0	6	5	6

Step 2: Calculate Available Resources

```
Total Resources:     R0=3,  R1=17,  R2=16,  R3=12
Total Allocated:     R0=2,  R1=12,  R2=14,  R3=12
                         (0+1+1+0+0=2, 1+2+3+6+0=12, 1+3+6+3+1=14,
0+1+5+2+4=12)
Available = Total - Allocated = [3-2, 17-12, 16-14, 12-12] = [1, 5, 2, 0]
```

Step 3: Calculate Need Matrix (Need = Max − Allocation)

Process	Need R0	Need R1	Need R2	Need R3
P0	0−0=0	2−1=1	1−1=0	0−0=0
P1	1−1=0	6−2=4	5−3=2	2−1=1
P2	2−1=1	3−3=0	6−6=0	6−5=1
P3	0−0=0	6−6=0	5−3=2	2−2=0
P4	0−0=0	6−0=6	5−1=4	6−4=2

Need Matrix Summary: P0=[0,1,0,0] P1=[0,4,2,1] P2=[1,0,0,1] P3=[0,0,2,0] P4=[0,6,4,2]

Step 4: Safety Algorithm — Find Safe Sequence

Initial: Work = Available = [1, 5, 2, 0] | Finish = [F, F, F, F, F]

Step	Process	Need	Work (before)	Need ≤ Work?	Work (after)
1	P0	[0,1,0,0]	[1,5,2,0]	YES ✓	[1,5,2,0]+[0,1,1,0]=[1,6,3,0]
2	P3	[0,0,2,0]	[1,6,3,0]	YES ✓	[1,6,3,0]+[0,6,3,2]=[1,12,6,2]
3	P2	[1,0,0,1]	[1,12,6,2]	YES ✓	[1,12,6,2]+[1,3,6,5]=[2,15,12,7]
4	P1	[0,4,2,1]	[2,15,12,7]	YES ✓	[2,15,12,7]+[1,2,3,1]=[3,17,15,8]
5	P4	[0,6,4,2]	[3,17,15,8]	YES ✓	[3,17,15,8]+[0,0,1,4]=[3,17,16,12]
✓ SYSTEM IS IN SAFE STATE | Safe Sequence: P0 → P3 → P2 → P1 → P4					

Conclusion: All Finish[i] = TRUE. All processes can complete. The system is in SAFE STATE with safe sequence: P0 → P3 → P2 → P1 → P4.

Q6. A system has 5 processes (P0–P4) and 3 resource types (A, B, C). Resource instances: A=5, B=5, C=3. Allocation and Maximum Claim given. Available = (1, 0, 1). Is system in safe state? Show safe sequence. [2+8]

Exam Ref: 2080 Chaitra, 2079 Ashwin

Answer:

Given Tables (3-Resource Variation)

Process	Alloc A	Alloc B	Alloc C	Max A	Max B	Max C	Need A	Need B
P0	0	3	1	7	5	3	7	2
P1	2	0	0	3	2	2	1	2
P2	3	0	2	9	0	2	6	0
P3	2	1	1	4	2	2	2	1
P4	0	0	2	5	3	3	5	3

Note: Need = Max − Allocation. C column of Need: P0=2-1=1, P1=2-0=2, P2=0-2... (use actual values from your exam paper)

Safety Check with Available = [1, 0, 1]

```
Work=[1,0,1]  Finish=[F,F,F,F,F]
Check P0: Need=[7,2,2] ≤ [1,0,1]? NO — skip
Check P1: Need=[1,2,2] ≤ [1,0,1]? NO — skip
Check P2: Need=[6,0,0] ≤ [1,0,1]? NO — skip
Check P3: Need=[2,1,1] ≤ [1,0,1]? NO — skip
Check P4: Need=[5,3,1] ≤ [1,0,1]? NO — skip
No process can proceed → UNSAFE STATE (check your specific exam paper values)
```

NOTE: The exact allocation/max values vary between exam papers (2079, 2080 variants). Always recompute Need = Max − Allocation and Available = Total − ΣAllocation for your specific question.

SECTION 4: Deadlock Detection & Recovery

Exam Ref: 2081 Ashwin, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2079 Ashwin

Q7. What is Deadlock? Explain the essential conditions for deadlock. How does the OS detect deadlock and what does it do after detection? [2+2+4]

Exam Ref: 2078 Chaitra, 2079 Chaitra

Answer:

Deadlock Detection

The OS periodically runs a detection algorithm to check if deadlock has occurred:

Single-Instance Resources: Wait-For Graph

For single-instance resource types, deadlock detection is done using a Wait-For Graph:

- Derived from RAG by removing resource nodes and collapsing edges
- Pi → Pj means Pi is waiting for a resource held by Pj
- Deadlock □ Cycle exists in the Wait-For Graph
- Cycle detection: Run DFS algorithm on the graph — $O(n^2)$ time

Multiple-Instance Resources: Detection Algorithm

Similar structure to Banker's Safety Algorithm:

```
1. Work = Available;  Finish[i] = false (if Allocation[i] ≠ 0), else true
2. Find i such that: Finish[i]==false AND Request[i] <= Work
3. Work = Work + Allocation[i];  Finish[i] = true;  go to step 2
4. If Finish[i]==false for ANY i → DEADLOCK exists
   Deadlocked processes = {Pi | Finish[i] == false}
```

How Often to Invoke Detection?

Frequency	Effect
Every resource request	Detects deadlock immediately; high overhead
Periodic (every N minutes)	Balance between overhead and response time
When CPU utilization drops below threshold	Practical — deadlock often causes low CPU use

Recovery from Deadlock

After deadlock is detected, the OS must recover. Two main approaches:

Method	How It Works	Issues
Process Termination (Abort)	Kill one or more processes in the deadlock cycle to break it. Choice: kill cheapest process (least computation done, least priority)	Work done by killed process is lost; may need user action

Abort All Deadlocked Processes	Kill ALL deadlocked processes — guaranteed to break cycle	Very expensive — all progress lost
Resource Preemption	Take a resource from a process (rollback it to safe state) and give to another. Select victim with minimum cost.	Starvation risk — same process may always be preempted
Rollback	Roll back the process that held the preempted resource to a checkpointed safe state	Requires checkpoint/rollback infrastructure

Victim Selection Criteria (for preemption): Priority of process, time run so far, resources held, resources needed to complete, number of rollbacks already done.

SECTION 5: Other Issues — Livelock, Starvation, Two-Phase Locking, Communication Deadlock

Exam Ref: 2081 Ashwin, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2077 Chaitra

Q8. Explain Two-Phase Locking, Communication Deadlock, Livelock, and Starvation as issues related to deadlock. [2+3+3+2]

Exam Ref: 2079 Chaitra, 2078 Chaitra, 2077 Chaitra

Answer:

Two-Phase Locking (Database Context)

Two-Phase Locking (2PL) is a protocol for transactions that prevents deadlock through controlled lock acquisition:

Phase 1 — Growing Phase: Transaction acquires ALL locks it needs. No lock is released during this phase.

Phase 2 — Shrinking Phase: Transaction releases locks one by one. No new locks are acquired during this phase.

- Advantage: If all transactions use 2PL, deadlock (in transaction context) cannot occur
- Disadvantage: All locks must be acquired before work begins → can cause starvation; reduces concurrency
- Example: In a bank transfer: lock account A, lock account B, debit A, credit B, release both

Communication Deadlock

Communication Deadlock occurs in distributed systems when processes are waiting for messages from each other:

- Process A is waiting to receive message from Process B
- Process B is waiting to receive message from Process A
- Neither can proceed — similar to resource deadlock but with messages as the 'resource'
- Causes: Lost messages, protocol design flaws, network partitions

Solution: Timeouts — if no message received within time T, assume deadlock and take recovery action. Use checksums to detect lost messages.

Livelock

Livelock is a situation where processes are ACTIVE (not blocked) but make NO PROGRESS because they keep responding to each other's actions:

Aspect	Details
Definition	Processes actively change state but cannot advance toward completion
Classic Example	Two people walking toward each other in a corridor, both step aside in the same direction, then both step back — forever
Code Example	Process A: while(!try_lock(R2)) { release(R1); } Process B does the same — they keep releasing and re-requesting
Detection	CPU usage is high (processes are running) but no useful work done
Solution	Introduce randomness: wait a random time before retrying (e.g., Ethernet CSMA/CD backoff algorithm)

Starvation

Starvation is when a process waits indefinitely because other processes are continually given preference:

- Cause: Priority-based scheduling where a low-priority process always loses to higher-priority ones
- Difference from Deadlock: In starvation, the process COULD run eventually; in deadlock, it CANNOT unless intervention
- Solution: Aging — gradually increase the priority of a waiting process over time
- Example: In SSTF disk scheduling, a distant cylinder request may wait forever while many nearby requests keep arriving

Integrated Deadlock Strategy

An integrated strategy combines multiple approaches for different resource types:

Resource Type	Recommended Strategy
Internal Resources (PCBs, etc.)	Prevention — order all resource requests
Main Memory	Prevention or preemption — swap process out
User Processes (files, locks)	Avoidance — Banker's Algorithm or Detection
Swappable Space	Preallocate all needed space — prevention of hold-and-wait

QUICK REVISION: Chapter 7 — Key Points Summary

Topic	Key Points	Expected Marks
Definition of Deadlock	Set of processes blocked in circular wait; each holds a resource and waits for another's resource	**2–3**
4 Necessary Conditions	Mutual Exclusion + Hold & Wait + No Preemption + Circular Wait — ALL must hold simultaneously	**3–4**
Deadlock vs Livelock vs Starvation	Deadlock=blocked; Livelock=active but no progress; Starvation=indefinite waiting due to priority	**2–4**
Deadlock Prevention	Attack one condition: sharable resources / request-all-at-once / preemption / ordered requests	**3–5**
Safe State & Banker's Algorithm	Safe if safe sequence exists. Data structures: Available, Max, Allocation, Need. Safety algorithm finds sequence.	**4–8**
Banker's Numerical	Need=Max−Alloc; Available=Total−ΣAlloc; Find safe sequence using work vector	**6–10**
Detection Algorithm	Similar to safety; if Finish[i]=false for any i → deadlock. Use Wait-For Graph for single instance.	**3–5**
Recovery	Abort processes OR preempt resources. Use aging to prevent starvation.	**2–4**
Two-Phase Locking	Growing phase (acquire) + Shrinking phase (release). Prevents circular wait in DB context.	**2–3**
Communication Deadlock	Distributed systems waiting for messages from each other. Solution: timeouts.	**2–3**

Best of Luck! | TU IOE BCT | CT 656 OS | Chapter 7: Deadlock